

GECKO

Technical Reference for Researchers

Processing Documentation & Data Transparency Report

GECKO v1.5.4 | August 2026

Sensorimotor Neuroscience Laboratory | Penn State University

Document Outline

This document describes every point at which GECKO modifies, filters, or transforms raw data. Its purpose is to ensure full transparency so researchers can accurately report methods and understand exactly what processing has been applied to their data.

- 1. Sentinel Value Cleaning — How invalid KINARM samples are identified and replaced
- 2. Channel Inspection Plots — What processing is applied when you view a channel
- 3. Interpolation Pipeline — Automatic gap filling, user-prompted strategies, and caching
- 4. Gaze Metric Calculations — Coordinate transforms, angular velocity, and Foveal Visual Radius (FVR)
- 5. Derived Kinematic Channels — Hand velocity, speed, acceleration, and force channels
- 6. Gaze Labeling & Export — Event codes, exported columns, and file formats
- 7. Output Files & Directory Structure — Where CSV, NPZ, and reference files are saved
- 8. Local Files & Stored Preferences — Session state, user preferences, crash logs
- 9. Units Reference — Units for all channels and computed metrics
- 10. Configurable Parameters — Table containing all of the configurable parameters in `user_prefs.py`

1. Sentinel Value Cleaning

KINARM systems do not use NaN to represent invalid or missing gaze data points. Instead, they record large sentinel values (typically ± 99.9 or greater) during periods of tracking loss, blinks, or sensor dropout. This program replaces these sentinel values with NaN (Not a Number) so that standard numerical operations can handle missing data correctly, and so that frame lengths aren't affected when outliers are removed.

1.1 When Sentinel Cleaning Occurs

Sentinel cleaning is applied at the following points:

- **Channel inspection:** When you select a channel to plot, any sample with $|\text{value}| \geq 99.9$ is replaced with NaN before the plot is drawn.

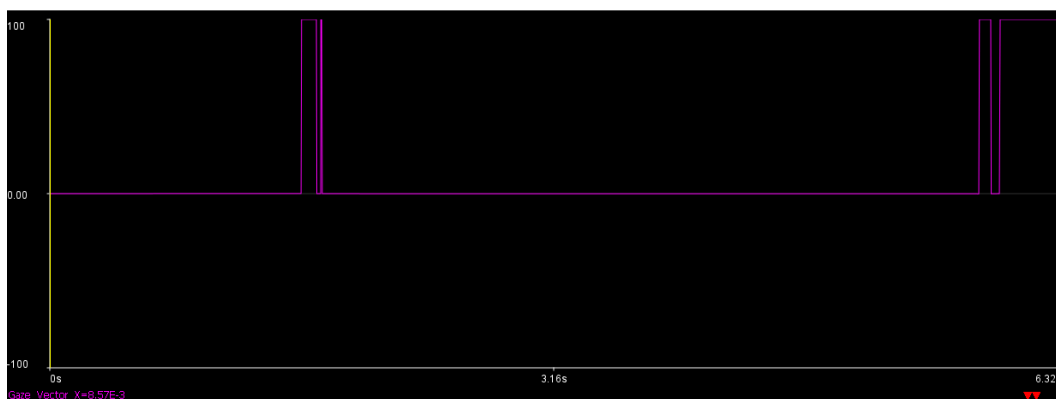


Figure 1: Raw KINARM file without sentinel removal. The large spikes at ± 99.9 represent invalid samples (Gaze_Vector_X channel).

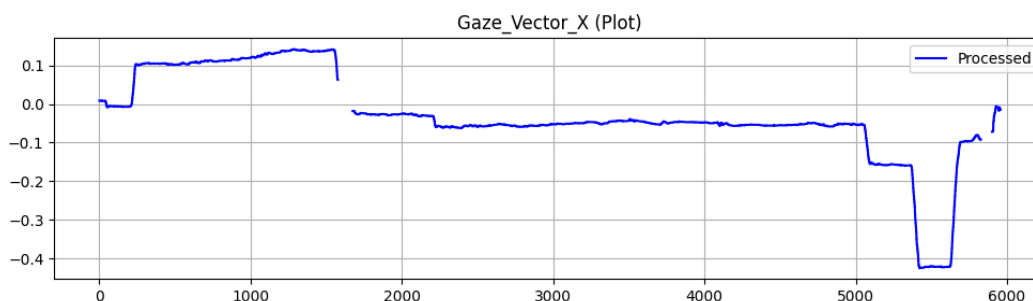


Figure 2: Same channel after sentinel cleaning. Invalid samples replaced with NaN, shown as gaps (Gaze_Vector_X channel).

- **Interpolation:** Before checking a channel for gaps, sentinel values are replaced with NaN. The interpolation system then treats these NaN regions as missing-data gaps.
- **Export pipeline:** All Gaze_* channels (excluding Gaze_TimeStamp) **are cleaned** of sentinel values before any calculations or labeling. Non-gaze channels **are not** sentinel-cleaned.
- **Buffer expansion:** In all three cases above, the invalid region is then expanded by a configurable buffer (`SENTINEL_BUFFER_FRAMES`, default 5 frames) on each side before gap detection runs, to catch tracker transition artifacts near a blink that don't individually cross the sentinel threshold but aren't valid gaze data either.

1.2 Channels Exempt from Sentinel Cleaning

Sentinel cleaning is applied *only to Gaze_** (e.g, Gaze_X, Gaze_Y, Gaze_PupilArea, Gaze_VectorX). All other channels are exported **as-is** without sentinel replacement. Gaze_TimeStamp is exempt even within the Gaze_* family, as it contains elapsed time in seconds and can legitimately exceed 99.9 for long trials.

Note: The ± 99.9 threshold is configurable and can be adjusted if your KINARM configuration produces legitimate values at or above this magnitude. The set of channels eligible for sentinel cleaning (currently: any Gaze_ channel, excluding Gaze_TimeStamp or any channel with "timestamp" in its name) is currently fixed in the application's source code, and configuration of such channels will be explored in the future.

2. Channel Inspection Plots

When you select channels from the Inspect Channel list and click “Inspect Selected Channels,” the following processing is applied before the plot is drawn:

2.1 Processing Steps

- **Step 1 — Sentinel cleaning:** For Gaze_* channels (excluding Gaze_TimeStamp), sentinel values ($|\text{value}| \geq 99.9$) are replaced with NaN, then expanded by the configurable sentinel buffer (Section 1.1). This must occur before interpolation—the NaN regions produced by sentinel cleaning are what the interpolation pipeline detects as gaps. Non-gaze channels are not sentinel-cleaned.
- **Step 2 — Interpolation prompt:** The application runs the interpolation pipeline on the cleaned channel data (see Section 3). Gaps of 50 frames or fewer are auto-filled linearly. Gaps larger than 50 frames prompt you to choose a strategy. If the channel has no gaps, this step is skipped.
- **Step 3 — Plot rendering:** The processed data is plotted. The y-axis label shows the channel name. The x-axis is in frames (sample index).

2.2 What Is Not Applied

No filtering is applied during channel inspection. The data you see in an inspection plot is the interpolated (or raw) signal with sentinels removed. **No low-pass filter, no smoothing, and no other processing is applied.** What you see is as close to raw data as possible, minus the sentinel-to-NaN conversion and any interpolation the user selects.

3. Interpolation Pipeline

Missing-data gaps (NaN regions after sentinel cleaning) are handled through a two-tier interpolation system. This runs whenever channels are inspected, when gaze data is prepared for labeling, or when channels are selected for export.

3.1 Gap Detection

After sentinel values are replaced with NaN, the invalid region is expanded by `SENTINEL_BUFFER_FRAMES` on each side (see Section 1.1). The system then identifies contiguous NaN regions (gaps) using connected-component labeling.

3.2 Automatic Small-Gap Filling

Gaps of 50 frames or fewer are automatically filled using linear interpolation without any user prompt. Linear interpolation draws a straight line between the last valid sample before the gap and the first valid sample after the gap. If a gap occurs at the very start or end of the signal (only one boundary exists), the nearest valid value is used to fill the gap (flat extrapolation).

Note: The 50-frame threshold is a default but is user-configurable (`user_prefs.py`). At a typical KINARM sampling rate of 1000 Hz, 50 frames correspond to 50 milliseconds of missing data.

3.3 User-Prompted Large-Gap Handling

Gaps larger than 50 frames trigger an interactive preview window showing three options:

- **Linear Interpolation:** Fills the gap with a straight line between boundary values. Appropriate when the signal likely transitioned smoothly during the gap.
- **Saccadic Interpolation:** Models a rapid transition during the first ~20% of the gap followed by a plateau near the end value. This mimics a saccadic eye movement profile using a sigmoid curve (configurable steepness). Appropriate for gaps during suspected saccades.
- **Leave as NaN:** Preserves the gap as-is. Downstream calculations will produce NaN for those frames.

Each large gap is handled individually—you are prompted once per gap, and each gap can receive a different strategy within the same trial and channel.

3.4 Interpolation Caching

Once you make an interpolation decision for a channel in a given trial, the result is saved for the duration of your session. If you inspect the same channel in the same trial again or export it later, the cached result is reused without re-prompting. Cache is cleared via the main menu option or reopening the program.

3.5 Timestamp Upsampling

The `Gaze_TimeStamp` channel receives special treatment. KINARM systems sometimes record timestamps at half the gaze sampling rate (e.g., 500 Hz timestamps for 1000 Hz gaze data),

producing consecutive repeated values like [1, 1, 2, 2, 3, 3]. Instead of interpolation, the system **upsamples** these by distributing repeated values into evenly spaced intervals: [1, 1.5, 2, 2.5, 3, 3.5].

4. Gaze Metric Calculations

The application computes several gaze metrics from raw Gaze_X and Gaze_Y channels. These represent the point-of-regard coordinates on the stimulus plane in meters.

4.1 Input Data

All gaze calculations use **Gaze_X** and **Gaze_Y** as inputs. These are the point-of-regard coordinates recorded by the KINARM eye tracker, representing where the participant is looking on the stimulus plane. The units are **meters**.

4.2 Eye Height Assumption

The eye is modeled as a point source located at a fixed height **H = 0.20 meters** (configurable in user_prefs.py) above the stimulus plane. This value is a lab-specific measurement and is used in all spherical coordinate and angular velocity computations.

4.3 Spherical Coordinate Transform

Gaze_X and Gaze_Y are converted from Cartesian coordinates to an eye-centered spherical coordinate system:

- **Rho (ρ):** Radial distance from the eye to the gaze point. Computed as $\sqrt{x'^2 + y'^2 + z'^2}$. Units: meters.
- **Theta (θ):** Azimuth angle in the horizontal plane. Computed as $\arctan(y'/x')$. Units: radians.
- **Phi (ϕ):** Elevation angle from the vertical axis. Computed as $\arccos(z' / \rho)$. Units: radians.

4.4 Angular Velocity

Angular velocity represents how fast the gaze direction is changing, measured in degrees per second. It is computed through the following steps ($x' = x$, $y' = y$, $z' = H$):

- **Step 1 — Time derivatives:** The Savitzky-Golay filter (default: window = 11, order = 3; configurable in user_prefs.py) is applied to compute the first time derivative of the eye-centered Cartesian coordinates ($\dot{x}$, $\dot{y}$, $\dot{z}$) and radial distance rate ($\dot{\rho}$). This filter smooths the derivative computation to reduce noise amplification.
- **Step 2 — Component rates:** The elevation rate ($\dot{\phi}$) and azimuth rate ($\dot{\theta}$) are computed from the Cartesian derivatives using the chain rule (Equations 4a and 4b from the reference paper).
- **Step 3 — Speed magnitude:** The angular speed is computed as $\sqrt{[(\dot{\theta} \cdot \sin\phi)^2 + \dot{\phi}^2]}$, then converted from radians/second to degrees/second. **Units: deg/s.**

4.5 Foveal Visual Radius (FVR)

FVR represents the radius of the foveal projection on the stimulus plane, which is computed as $\rho(\tan(\delta/2)) * \csc(\epsilon)$. Here, ρ is the viewing distance, δ is the foveal cone angle (defined in the reference paper as $3^\circ + \text{mean calibration error}$; default: 5.0° , configurable in `user_prefs.py`), and ϵ is the angle between the gaze direction vector $[x', y', z']$ and the stimulus plane normal. Units: meters.

4.6 Filters Applied During Calculations

All gaze metric calculations (whether triggered from the GUI buttons or computed during export) apply the same filtering pipeline:

- **4th-order Butterworth low-pass filter (effective order 8 due to zero-phase forward-backward filtering via `filtfilt`) at 20 Hz cutoff** is applied to both `Gaze_X` and `Gaze_Y` before any calculations. This removes high-frequency noise from the gaze signal. NaN values are temporarily linearly interpolated to allow the filter to run, then restored afterward. (This means the filter has no effect on the shape of NaN gaps in the output—the NaN positions are preserved exactly.)
- **Savitzky-Golay filter** (default: `window = 11`, `order = 3`; configurable in `user_prefs.py`) is used internally to compute time derivatives for angular velocity (Section 4.4).

5. Derived Kinematic Channels

The application computes additional kinematic channels from the raw position data recorded by the KINARM system. These channels do not exist in the .kinarm file and are calculated on demand. The Hand Positions are converted into mm (from meters) and all the other channels are calculated in mm/s and mm/s².

5.1 Available Derived Channels

Channel Name	Computed From	Unit
Right_HandX / Left_HandX	Right/Left hand X position (raw)	mm
Right_HandY / Left_HandY	Right/Left hand Y position (raw)	mm
Right_HandVelX / Left_HandVelX	1st derivative of X position	mm/s
Right_HandVelY / Left_HandVelY	1st derivative of Y position	mm/s
Right_HandSpeed / Left_HandSpeed	Magnitude of velocity vector	mm/s
Right_HandAccX / Left_HandAccX	2nd derivative of X position	mm/s ²
Right_HandAccY / Left_HandAccY	2nd derivative of Y position	mm/s ²
Right_HandCmdFX / Left_HandCmdFX	Motor 1 torque command (raw)	Torque
Right_HandCmdFY / Left_HandCmdFY	Motor 2 torque command (raw)	Torque
FP1_CoPX	Force plate moment X (raw)	mm
FP1_CoPY	Force plate moment Y (raw)	mm

5.2 Filters Applied to Derived Channels

Velocity and acceleration channels are computed using numerical differentiation (numpy.gradient) with respect to time ($dt = 1/\text{frame_rate}$), where frame_Rate is read from the trial metadata. No filtering is applied to the derivative output. Position channels, torque/force channels, and force plate channels are passed through as raw values with no filtering.

5.3 Lab-Specific Availability

Derived channels are computed from raw position data for **Right_Hand** and **Left_Hand**. If your .kinarm file does not contain these position labels, the derived channels will not appear in the channel list. Force plate channels (FP1_CoPX, FP1_CoPY) similarly require the corresponding raw kinematics channels (FP1_MX, FP1_MY) to be present. Channels that cannot be computed for the current file are silently skipped.

6. Gaze Labeling & Export

6.1 Gaze Event Codes

During manual labeling, you annotate time periods as fixations, pursuits, or saccades. These are converted to a per-frame integer code in the exported data:

Code	Meaning
0	Unlabeled / Target appearance instant
1	Saccade
2	Pursuit
3	Fixation
9	Bad trial (entire trial marked unusable)

If a trial is marked as “bad,” all frames are assigned code 9 regardless of any labeling performed.

6.2 Data Processing During Export

When you run the labeling/export pipeline, the following processing is applied:

- **Interpolation:** Before the labeling interface opens, all channels selected for both labelling display and for export are interpolated in a single pass. This means all interpolation decisions—for both the channels you will see during labeling and the channels that will appear in the exported CSV—are made upfront before labeling begins. Users have full control over how each channel's gaps are handled: gaps of 50 frames or fewer are auto-filled linearly, while gaps larger than 50 frames present an interactive preview allowing you to choose linear interpolation, saccadic interpolation, or leave the gap as NaN. If you choose to leave a gap as NaN, those frames will appear as NaN in the exported CSV and/or appear as gaps in the labeller plot.
- **All Gaze_* channels (excluding Gaze_TimeStamp):** Sentinel values (± 99.9) are replaced with NaN. Every other channel, such as xT, yT are not sentinel-cleaned.
- **Gaze metrics (Rho, Theta, Phi, Angular Velocity, FVR):** Computed from the cleaned Gaze_X/Gaze_Y after linearly filling all NaN gaps. The same 20 Hz low-pass filter described in Section 4.6 is applied before computation. These computed columns are never interpolated—only the raw input channels are. Only occurs if a metric is selected to export.

6.3 CSV Format

Each exported CSV file contains one row per frame with the following columns:

- **Frame:** 0-based frame index.
- **Gaze_Events:** Integer gaze event code (see table above).
- **Selected channels:** All channels you chose in the export selection list, in the order selected.

- **Event columns:** If you select event markers for export (e.g., TARGET_ON), they appear as additional columns. Row 0 contains a semicolon-separated list of frame numbers where that event occurred; all other rows are empty.

6.4 NPZ Format

A compressed NumPy archive (.npz) is also saved alongside each CSV to provide a compact alternative for storage, as CSV files can become large when exporting many channels across long trials. NPZ files are stored in an `npz/` subfolder to reduce clutter.

7. Output Files & Directory Structure

7.1 Default Save Location

By default, all exports are saved to a folder named `gaze_labels` on your Desktop:

```
~/Desktop/gaze_labels/<your_kinarm_file.kinarm>/
```

You can change this location at any time using the Save Location menu in the GUI. The custom location is remembered across sessions.

7.2 Directory Layout

Within your chosen save location, a subfolder is created for each .kinarm file:

File	Description
Trial1.TP1.C1.csv	Exported data for Trial 1, Task Protocol 1, Count 1
Trial1.TP1.C1.npz	(in npz/ subfolder) Compressed NumPy version
Target_Table.csv	Workspace target definitions (positions, radii, colors)
TP_Table.csv	Task protocol parameters (timing, delays, targets)
Trial_Notes.csv	Session-based trial marks and free-text notes
session_state.json	GUI state for session resume (see Section 8)

7.3 File Naming Convention

Trial data files follow the pattern: `Trial{X}.TP{Y}.C{Z}.csv` where X is the trial number (1-based), Y is the Task Protocol number, and Z is the repeat count for that trial within the protocol.

8. Local Files & Stored Preferences

The application stores several files on your computer to preserve your settings and session state between uses. No data is transmitted externally.

8.1 User Preferences File

A JSON file is stored in your user configuration directory:

`~/.config/KinarmDataExplorer/user_prefs.json` on macOS/Linux, or
`%APPDATA%\KinarmDataExplorer\user_prefs.json` on Windows.

This file stores:

- **Export defaults:** The channels you last selected for export.
- **Marker defaults:** The event markers you last selected for plotting.
- **Custom save location:** Your chosen output directory, if changed from the default.
- **Label order:** Your preferred order for labeling gaze events (fixation/pursuit/saccade sequence).

This file is human-readable JSON and can be manually edited or deleted. Deleting it resets all preferences to defaults.

8.2 Session State File

For each .kinarm file you open, a `session_state.json` is saved in the output directory alongside your exports. It records:

- Which trial was selected when you last closed the file
- Your channel filter/search text and channel inspection selections

Export channel selections and event marker selections are stored in `user_prefs.json` (Section 8.1), not in `session_state.json`. These persist globally across all files rather than per-file.

When you reopen the same .kinarm file, the application will prompt you to resume where you left off—returning to the previously selected trial. A second prompt asks whether to restore your previous channel inspection selections. If you decline either prompt, the application opens normally without restoring that state. Session state can be cleared at any time via the Clear Cache menu option.

8.3 Trial Marks & Notes

Trial quality marks (Good, Bad, Review) and free-text notes are saved as `Trial_Notes.csv` in the output directory. This file is updated whenever you change a trial's mark or notes. A separate `Trial_Marks_and_Notes.csv` is generated during export, containing marks and notes for all trials with annotations.

8.4 Crash Log

If the application crashes unexpectedly, a crash report is written to:

- `~/Desktop/kinarm_crash_report.txt` (preferred location)
- `~/kinarm_crash_report.txt` (fallback if Desktop is unavailable)

8.5 Save Location and Session Portability

Session state and trial notes are stored inside the export directory, not alongside the `.kinarm` file. If you change your save location, the application will not find your previous session state or notes — it will behave as if you are opening the file for the first time. Your previously exported data remains in the original location.

Sharing session data with collaborators currently requires manually copying export folders and ensuring they are placed in the correct save location. A future update will simplify this system to allow for easier collaboration and better portability.

9. Units Reference

The following table summarizes the units for all channels and computed metrics in the application:

Channel / Metric	Unit	Notes
Gaze_X, Gaze_Y	meters	Point-of-regard on stimulus plane
Rho (ρ)	meters	Eye-to-gaze radial distance
Theta (θ)	radians	Azimuth angle
Phi (ϕ)	radians	Elevation angle
Angular Velocity	deg/s	Gaze angular speed magnitude
FVR	meters	Foveal Visual Radius
Right/Left_HandX, HandY	mm	Hand position from KINARM robot
Right/Left_HandVelX, VelY	mm/s	Hand velocity
Right/Left_HandSpeed	mm/s	Hand speed magnitude
Right/Left_HandAccX, AccY	mm/s ²	Hand acceleration
Right/Left_HandCmdFX, CmdFY	Torque	Motor torque commands
FP1_CoPX, FP1_CoPY	mm	Force plate center of pressure
Gaze_TimeStamp	seconds	Upsampled if recorded at half rate
Frame	index (0-based)	Sample index in exported data
Gaze_Events	integer code	0/1/2/3/9 (see Section 6.1)

10. Configurable Parameters

The following table summarizes the current default parameters that are adjustable in `user_prefs.py`.

Parameter	Default	Description
KINARM_INVALID_ABS_THRESHOLD	99.9	Sentinel value threshold for NaN replacement
AUTO_INTERP_THRESHOLD_FRAMES	50	Gap size below which linear interpolation is applied automatically
DEFAULT_EYE_HEIGHT_M	0.20	Eye height above stimulus plane (meters)
DEFAULT_VISUAL_ANGLE_DEG	5.0	Foveal cone angle δ for FVR computation (degrees)
DEFAULT_GAZE_LOWPASS_CUTOFF_HZ	20	Butterworth cutoff for gaze metric preprocessing (Hz)
DEFAULT_GAZE_LOWPASS_ORDER	4	Butterworth filter order (effective order doubled by <code>filtfilt</code>)
DEFAULT_SAVGOL_WINDOW	11	Savitzky-Golay window length for angular velocity derivatives
DEFAULT_SAVGOL_POLYORDER	3	Savitzky-Golay polynomial order
SACCADIC_TRANSITION_FRACTION	0.2	Fraction of gap used for sigmoid transition in saccadic interpolation
SACCADIC_SIGMOID_STEEPNESS	10.0	Sigmoid steepness for saccadic interpolation
MAX_LABELER_CHANNELS	6	Maximum overlay channels in gaze labeler (including Gaze_X/Y)
SENTINEL_BUFFER_FRAMES	5	Frames discarded on each side of a detected sentinel gap to remove tracker transition noise
MPL_BACKEND	Qt5Agg	Matplotlib rendering backend. Change to TkAgg if Qt5/PyQt5 is unavailable

This document was prepared for peer review and publication transparency. For questions or clarifications, contact the Sensorimotor Neuroscience Laboratory at Penn State University.